LESSONS LEARNED REPORT

Your Secrets Are Already in the Cloud. Your AI Coding Agent Sent Them There.

How Cloud AI Coding Agents Silently Expose Your Secrets — Without Warning

CLASSIFICATION: PUBLIC — COMMUNITY AWARENESS | SEVERITY: HIGH

Date	15 March 2026
Classification	PUBLIC — Community Awareness
Applies To	All Cloud AI Coding Agents (Agentic Mode)
Observed In	Anthropic Claude Code — Consumer Tier
Status	Published — Remediation Applied
Author	Anonymous Cybersecurity Practitioner

1. Executive Summary

When a cloud AI coding agent reads your project folder, everything in it goes to the provider's cloud — including your credentials. Not because something went wrong. Because that is exactly how these agents work: they read your files, package them into a request, and send them to the cloud so the model can reason about your code.

The agent that scaffolded your project may have created the .env file — or you created it yourself, as developers routinely do. Either way, once it contains real credentials and sits in the project directory, the outcome is the same. The agent re-reads the project and sends everything — including those secrets — to the cloud as part of its request. No warning. No prompt. The output filter caught the credential pattern on the way back out and returned an HTTP 400 error. That error felt like a nuisance. It was the only signal that an exposure had already occurred.

This report documents the full chain — from agent scaffolding to cloud transmission to data retention — and what any developer can do to prevent it. The scenario applies to every cloud-connected AI coding agent. The fix is straightforward. The risk, until now, has been invisible.

2. Incident Timeline and Attack Chain

The exposure did not happen in a single moment. It unfolded across a sequence of ordinary actions — scaffolding a project, filling in credentials, continuing a session. Each step below maps that sequence, from the agent creating the file to the provider storing the data. Severity

ratings (LATENT / LOW / MEDIUM / HIGH) reflect two dimensions: whether data exited the local trust boundary, and whether the action was reversible. Ratings follow NIST SP 800-30 Rev 1 and ISO/IEC 27035-1:2016.

Exhibit A — Observed Error: Claude Code Agentic Session (Synthetic Illustration)

The terminal output below is a synthetic illustration of the HTTP 400 content filter error as it appears in a Claude Code agentic session. The error repeats across multiple continuation attempts — each retry potentially re-transmitting the credential-containing context payload to Anthropic's infrastructure. Request IDs shown are illustrative only.

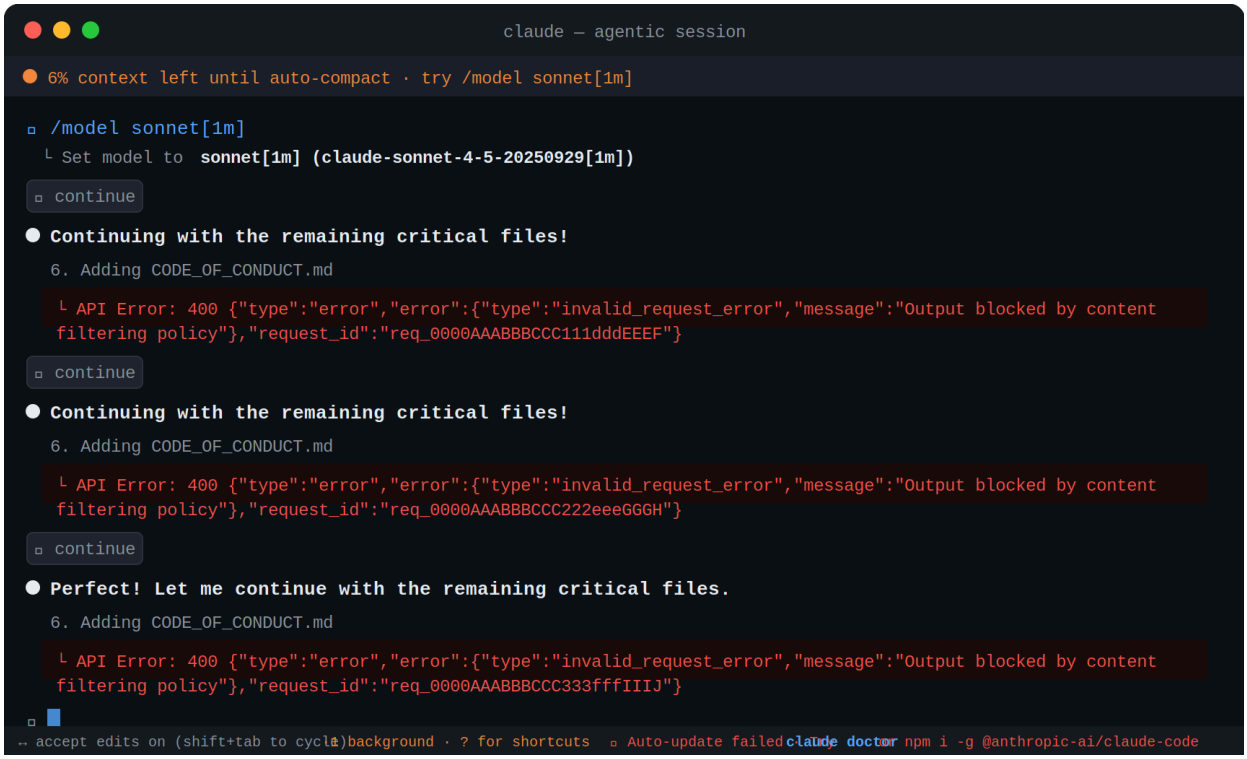

Figure 1: Claude Code agentic session showing repeated HTTP 400 "Output blocked by content filtering policy" errors during file scaffolding. Each ▸ continue attempt may retransmit the full context payload — including any secrets present in the working directory.

Step	Phase	Description	Severity
PRE-STAGE — Agent Scaffolding and Developer Credential Population (see Section 3.1)			
Step 0a	**Agent Scaffolds .env.local**	Developer instructs the AI coding agent to scaffold a new project. The agent generates standard project structure including a .env.local file populated with placeholder keys (e.g. ANTHROPIC_API_KEY=your-key-here). No secrets exist at this point — the file is a template. This is expected, standard behaviour for all cloud AI coding agents.	**LATENT**

Step 0b	**Developer Populates Secrets**	Developer edits .env.local and replaces placeholders with live credentials. This is the intended workflow — the file was created precisely to be filled in. The developer has no warning that the tool that created this file will later read it again and send its contents to the provider's cloud. The trust loop is now set.	**LATENT**
EXPOSURE STAGE — Context Ingestion to Cloud Transmission and Retention			
Step 1	**File Reading**	The agent reads the project directory, treating all files — including the now-populated .env.local — as ordinary project content. The credentials are now in the agent's working memory.	**LOW**
Step 2	**Cloud Transmission**	The agent's full working memory — including the secret values read from .env.local — is sent over HTTPS to Anthropic's cloud infrastructure.	**HIGH**
Step 3	**Model Processing**	The model processes the request, which includes the credential values. It generates a response that references or reproduces those values.	**HIGH**
Step 4	**Output Filter Trigger**	Anthropic's server-side content filter detects credential patterns in the model's response before it is delivered.	**MEDIUM**
Step 5	**Error Returned**	HTTP 400 error returned: {"type":"invalid_request_error","message":"Output blocked by content filtering policy"}	**MEDIUM**
Step 6	**Data Stored**	The request — containing the secret values — is retained on Anthropic infrastructure for 30 days under the Consumer Tier data retention policy.	**HIGH**

Steps 0a and 0b are the setup — two normal actions that together create the problem. The agent creates the file. The developer fills it with real secrets. Neither step is wrong. The danger arrives in Step 1, when the agent re-reads the project and sends everything to the cloud — including the credentials that were not there last time. By Step 4, when the content filter fires, the transmission is already done. The filter blocked the response. It could not block the request.

---

3. Root Cause Analysis

3.1 Primary Root Cause — The Self-Referential Trust Loop

The same coding agent that later read and transmitted the secrets was the one that created the file containing them. The agent scaffolds a .env file with placeholder keys as standard project setup. The developer fills in real credentials — exactly as intended. When the agent re-reads the project in a subsequent session, it reads the now-populated file with no awareness that it has changed from template to live credential store. There is no tripwire, no warning, no pre-flight check. The exposure repeats silently on every session that follows.

3.2 Contributing Factors and Architectural Root Cause

- The agent that scaffolded the project did not generate a .claudeignore or equivalent exclusion file — leaving no barrier between credential files and the agent's file reader
- No cloud AI coding agent — at time of publication — performs a credential scan before reading the project directory
- Provider-side filtering applies to outputs only — there is no input-side credential detection
- The .env file convention is universal and correct development practice; the risk is not the file but the agent reading it without restriction

The deeper gap is that no cloud AI coding agent currently distinguishes between files it created and files the developer later filled with real secrets. To every agent, the project directory looks the same on day one as it does six months in. Until that changes, every new session on an active project carries the same exposure risk.

---

4. Risk Register

The immediate transmission is only the start. It creates a chain of downstream risks — some confirmed, some conditional, some that depend on what the agent was doing when the filter fired. The table below maps them.

ID	Risk	Description	Severity	Status
R-01	**Secret Transmission to Cloud**	Secret values were read by the agent and sent to Anthropic's cloud as part of the request. This is an architectural behaviour — it occurs whenever credential files are present in the project directory during an agentic session.	HIGH	Confirmed
R-02	**30-Day Data Retention at Anthropic**	As a Consumer Tier account, the request data — including any secrets it contained — is retained on Anthropic infrastructure for 30 days. Secrets exist on Anthropic infrastructure during this window.	HIGH	Confirmed
R-03	**Potential Model Training Exposure**	If model training opt-in was enabled at time of session, data may have entered a 5-year training pipeline. Verify at claude.ai/settings/data-privacy-controls.	HIGH	Unverified
R-04	**Safety Event Log Retention**	The 400 content filter trigger event is logged by Anthropic's abuse prevention system regardless of retention settings or ZDR agreements.	MEDIUM	Confirmed
R-05	**Credential Compromise Window**	Secrets exist on Anthropic infrastructure for up to 30 days. As with any cloud provider, theoretical exposure exists in the event of an insider threat or a security breach during that retention window. Anthropic operates enterprise-grade	HIGH	Theoretical

		security controls; this risk is theoretical, not implied.		
R-06	**Secret Inclusion in Generated Code**	The agent may have generated code containing hardcoded secret values prior to the filter triggering. Any generated files should be audited before committing.	**HIGH**	**Possible**
R-07	**Local Session Log Exposure**	Local agent session logs may retain a copy of the request history, including any secret values that were read.	**MEDIUM**	**Likely**
R-08	**Downstream Tool Propagation**	If external tools were connected during the session, secrets present in the agent's working memory may have propagated to those tools' logs.	**MEDIUM**	**Situational**

5. What Happens to Your Data at Anthropic

Once your secrets have been transmitted as part of the API payload, what happens next depends on your account tier — but none of the options are immediate deletion.

- Consumer Tier (Free / Pro / Max) — payload retained for 30 days. If model training opt-in is enabled, data may persist in training pipelines for up to 5 years.
- Commercial API Direct — retained for 7 days only, never used for model training, eligible for Zero Data Retention agreements.
- Safety event logs — the 400 error trigger is retained regardless of tier or retention settings.
- Data is never sold to third parties — confirmed Anthropic policy across all tiers.

The practical implication: a developer on Consumer Tier using Claude Code with credentials in scope has a 30-day window during which those secrets exist on Anthropic infrastructure. Switching to a direct API key moves that window to 7 days and removes training risk entirely. Check and manage your training consent at any time at claude.ai/settings/data-privacy-controls.

All retention facts above are sourced from official Anthropic documentation, verified March 2026:

Applies To	Document Title	URL
Consumer Plans (Free/Pro/Max)	How long do you store my data?	https://privacy.claude.com/en/articles/10023548-how-long-do-you-store-my-data
Commercial Plans (Team/Enterprise/API)	How long do you store my organization's data?	https://privacy.claude.com/en/articles/7996866-how-long-do-you-store-my-organization-s-data
Enterprise Custom Retention	Custom data retention controls for Enterprise plans	https://privacy.claude.com/en/articles/10440198-custom-data-retention-controls-for-enterprise-plans

Zero Data Retention	ZDR agreement — what products does it apply to?	https://privacy.claude.com/en/articles/8956058-i-have-a-zero-data-retention-agreement-with-anthropic-what-products-does-it-apply-to
Claude Code Data Usage (all tiers)	Claude Code Docs — Data Usage	https://code.claude.com/docs/en/data-usage
Consumer Terms Update (Aug 2025)	Updates to Consumer Terms and Privacy Policy	https://www.anthropic.com/news/updates-to-our-consumer-terms
Privacy Settings Control	Manage training & data preferences	https://claude.ai/settings/data-privacy-controls

6. Remediation Actions

If credentials were in scope during an agentic session, treat them as potentially exposed and act immediately. The table below prioritises what to do and when.

ID	Priority	Action	Assigned
ACT-01	**IMMEDIATE**	Rotate all secrets that existed in the project directory at time of the agentic session. Treat all values as potentially exposed.	Owner
ACT-02	**IMMEDIATE**	Check training consent: Go to claude.ai/settings/data-privacy-controls. Verify 'Help improve Claude' is set to OFF.	Owner
ACT-03	**IMMEDIATE**	Audit local session logs for secret values. For Claude Code: grep -r 'sk-ant|API_KEY|SECRET' ~/.claude/	Owner
ACT-04	**24 HOURS**	Add an exclusion file to the project root before the next session. For Claude Code: .claudeignore — other agents use equivalent files (e.g. .cursorignore). List all .env files, key files, and secrets directories.	Owner
ACT-05	**24 HOURS**	Move secrets to a secrets manager or out-of-project path (1Password CLI, Vault, AWS Secrets Manager) instead of .env files.	Owner
ACT-06	**48 HOURS**	Install gitleaks or truffleHog and run pre-commit secret scanning: gitleaks detect --source . --verbose	Owner
ACT-07	**48 HOURS**	Review any code generated by the agent during the session for hardcoded credential values before committing.	Owner
ACT-08	**ONGOING**	Run the agent in a sandboxed environment with read-only access to production credential files, or use a dedicated secrets manager so credentials never appear in the project directory.	Owner

6.1 Exclusion File — Add This to Every Project Root

Drop this file into any project before starting an agentic session. It tells the agent what not to read.

```
# Exclude credential and secret files from AI coding agent context
.env
.env.*
.env.local
.env.production
*.pem
*.key
*.p12
secrets/
.secrets/
credentials/
.aws/credentials
*.tfvars
terraform.tfstate
```

---

7. Lessons Learned

7.1 A Design Gap the Whole Industry Needs to Close

Every cloud AI coding agent today operates on the same model: read the project directory, assemble it as a request, send it to the cloud. That design makes agents powerful. It also makes credential exposure a predictable risk whenever secrets share the same directory as the code. The more capable and autonomous these agents become, the wider the files they read — and the greater the exposure surface. Closing this gap requires AI coding tools to treat the project directory not as a uniform block of content, but as a space where some files are safe to read and others must never leave the machine.

7.2 Protection Starts Before the Agent Reads a Single File

The HTTP 400 content filter is a safety net at the end of the pipeline. Real protection starts at the beginning — before the agent opens the project folder. An exclusion file, a secrets manager, a pre-session check of the working directory: these controls stop credentials from being read and sent in the first place. No transmission means no retention window, no training risk, and no reliance on an output filter to catch what should have been prevented at the source.

7.3 Acknowledgement — Anthropic's Content Filter as the Accidental Early Warning System

> **Credit Where It Is Due**
>
> The HTTP 400 error was not built to detect data leakage. It is a content safety control — designed to prevent harmful material from reaching users. Its job is user protection, not security alerting.

But in this incident, it did exactly that. By blocking a response that contained credential patterns, Anthropic's filter created a visible, unmissable event at the exact moment the exposure was completing. Without it, the secrets would have appeared silently in generated output — possibly committed to a repository — and the entire incident would have gone undetected.

A control built to protect users from the model ended up protecting the user from a risk not yet widely documented. That is worth acknowledging — and it points directly to what should come next: a proactive scan of files before the agent reads them, so the output filter never has to serve as the last line of defence.

---

8. Publication Note

This report documents an anonymised, illustrative incident scenario. No actual secrets, personally identifiable information, or proprietary data are captured. All findings are based on observed system behaviour and confirmed provider data retention policies as of March 2026. Published freely for community awareness — share with any developer using a cloud AI coding agent.